

Othello

Grégory BONZON, Rémi BOUSSION, Gabriel RINGUET, Matthieu Vigier-Lafosse,
Arthur VOISIN

Présentation du jeu :

- Jeu de stratégie abstrait à information parfaite
- Deux joueurs (Noir et Blanc) sur un tablier huit par huit
- Mécanique principale : encadrer les pions adverses pour les retourner
- L'objectif est de posséder la majorité des pions



IA & Heuristiques

Signature : AlphaBeta

Contexte : Algorithme récursif principal de l'IA (Package Model).

Entrées :

- **etat** (GameState) : Configuration actuelle du plateau.
- **profondeur** (Entier) : Nombre de coups à anticiper avant l'évaluation.
- **alpha** (Flottant) : La meilleure valeur (maximale) trouvée jusqu'à présent pour le joueur MAX.
- **beta** (Flottant) : La meilleure valeur (minimale) trouvée jusqu'à présent pour le joueur MIN.
- **joueur_max** (Booléen) : **VRAI** si c'est au tour de l'IA (qui cherche à maximiser le score).

Sortie :

- (Flottant/Entier) : Le score heuristique estimé pour cette branche de l'arbre de jeu.

Rôle : Parcourt l'arbre des coups possibles pour déterminer la meilleure issue, en coupant les branches inutiles (élagage) pour optimiser le temps de calcul.

Fonctions d'évaluation du plateau :

- - Fonction 1 : évaluation positionnelle
- - Fonction 2 : évaluation basée sur la mobilité
- - Fonction 3 : évaluation mixte et dynamique

MCTS

Contexte : Algorithme d'exploration stochastique principal de l'IA (Package `Model`).

Entrées :

- `etat` (GameState) : Configuration actuelle du plateau à la racine de l'arbre.
- `temps_limite` (Flottant) : Temps maximum alloué pour la construction de l'arbre (en secondes).
- `strategie` (Chaine/Enum) : La méthode de sélection utilisée pour traverser l'arbre ('UCT' ou 'ML').
- `modele` (Objet) : Le modèle prédictif entraîné (requis uniquement si la stratégie 'ML' est sélectionnée).

Sortie :

- (Coup) : L'action considérée comme la meilleure, correspondant au nœud enfant de la racine ayant accumulé le plus grand nombre de visites.

Rôle : Construit itérativement un arbre de recherche asymétrique en enchaînant quatre phases (sélection, expansion, simulation, rétropropagation) pour converger statistiquement vers le coup le plus robuste avant l'écoulement du temps.

[Exemple](#)

Compute_Flips

Algorithme de retournement (Othello/Reversi)

```

1  FONCTION compute_flips(own_board, enemy_board, x, y)
2
3  // 1. Initialisation
4  masque_retournements_total = 0
5
6  // Liste des 8 directions (dx, dy)
7  directions = [(-1, -1), (-1, 0), (-1, 1),
8                (0, -1),      (0, 1),
9                (1, -1),  (1, 0),  (1, 1)]
10
11 // 2. Vérification préalable
12 SI la case (x, y) n'est pas vide OU hors plateau ALORS
13     RETOURNER 0
14 FIN SI
15
16 // 3. Boucle principale
17 POUR CHAQUE (dx, dy) DANS directions FAIRE :
18
19     x_courant = x + dx
20     y_courant = y + dy
21     masque_direction = 0
22
23     TANT QUE est_dans_grille(x_courant, y_courant) FAIRE :
```

```

24
25     index = (y_courant * 8) + x_courant
26     bit_mask = 1 << index // Décalage binaire
27
28     // CAS A : Pion ADVERSE
29     SI (enemy_board ET bit_mask) != 0 ALORS
30         masque_direction = masque_direction OU bit_mask
31         x_courant = x_courant + dx
32         y_courant = y_courant + dy
33
34     // CAS B : Pion À NOUS
35     SINON SI (own_board ET bit_mask) != 0 ALORS
36         masque_retournements_total = masque_retournements_total
37         OU masque_direction
38         ARRÊTER // Passer à la direction suivante
39
40     // CAS C : Case VIDE
41     SINON
42         ARRÊTER
43     FIN SI
44
45     FIN TANT QUE
46
47     FIN POUR
48
49     RETOURNER masque_retournements_total
50 FIN FONCTION
```

Signature : compute_flips

Entrées :

- own_board (Bitboard) : Plateau du joueur actuel.
- enemy_board (Bitboard) : Plateau de l'adversaire.
- x, y (Entiers) : Coordonnées du coup joué.

Sortie :

- (Bitboard) : Un masque représentant tous les pions retournés.

Rôle : Calcule les pions adverses qui seront capturés si le joueur pose un pion en (x, y) .

Exemple

Architecture du Projet

Common

ConfigManager

load_config()
set_config(param,value)
save_game(filepath, state)
load_game(filepath)
create_default_config()
usage(str) -> str est l'option
valid_options_check()

ArgParser

parse() -> créé un
dictionnaire config

I18nManager

i18_init() -> initialise gettext
+ fichier de traductions

Orchestrator

AppOrchestrator

run()
quit()

BitboardOps

set_disk(bitboard, index)
get_disk(bitboard, index) : Manipula
basique des bits.
popcount(bitboard) Compter le nom
de bits à 1 (pour calculer le score).
shift_direction(bitboard, direction) :
Pour décaler les bits.

GameSession

start_new_game(player1,player2,color,size,mode)
load_default_board(size)
command_loop() -> interroge la vue
pour récupérer l'input (commande)
donc soit un readline() pour cli
soit des boutons pour gui
command_help()

Interface GameMode

set_mode_settings(config)

BlitzController

pause(gameclock)
resume(gameclock)
set_time_limit()

ContestController

run(file)

Game Engine

GameState

undo_stack : stack
redo_stack : stack
boards : int
white_board : int
black_board : int
get_board()
compute_score()
restore_previous()
restore_board(undostack,
redostack)
switch_turns()
show_board()

GameRules

is_valid_move(bitboard, color, x, y)

get_legal_moves(bitboard, color)

compute_flips(bitboard, color, x, y)

move_parser(str) -> lecture de la ligne
de commande exemple X e4

is_game_over()

MoveAdvisor

get_advice(board)

GameClock

update_clocks(whichp)

pause_clock(whichp)

show_time()

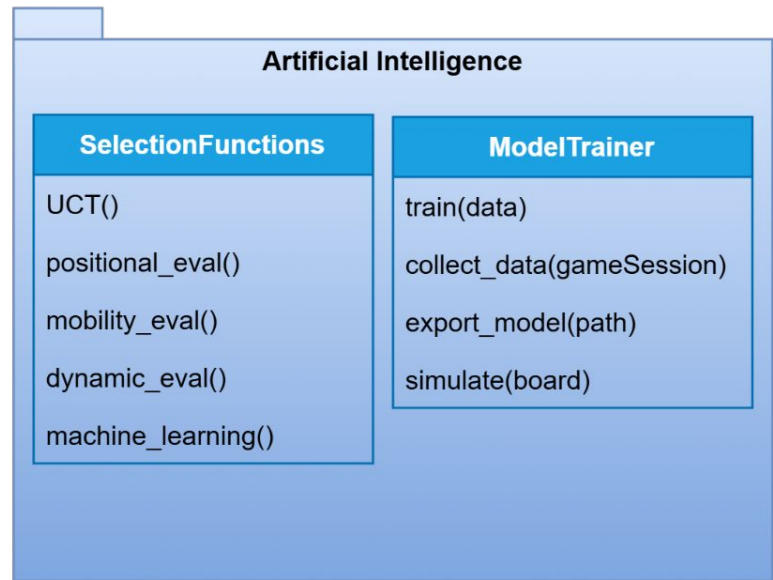
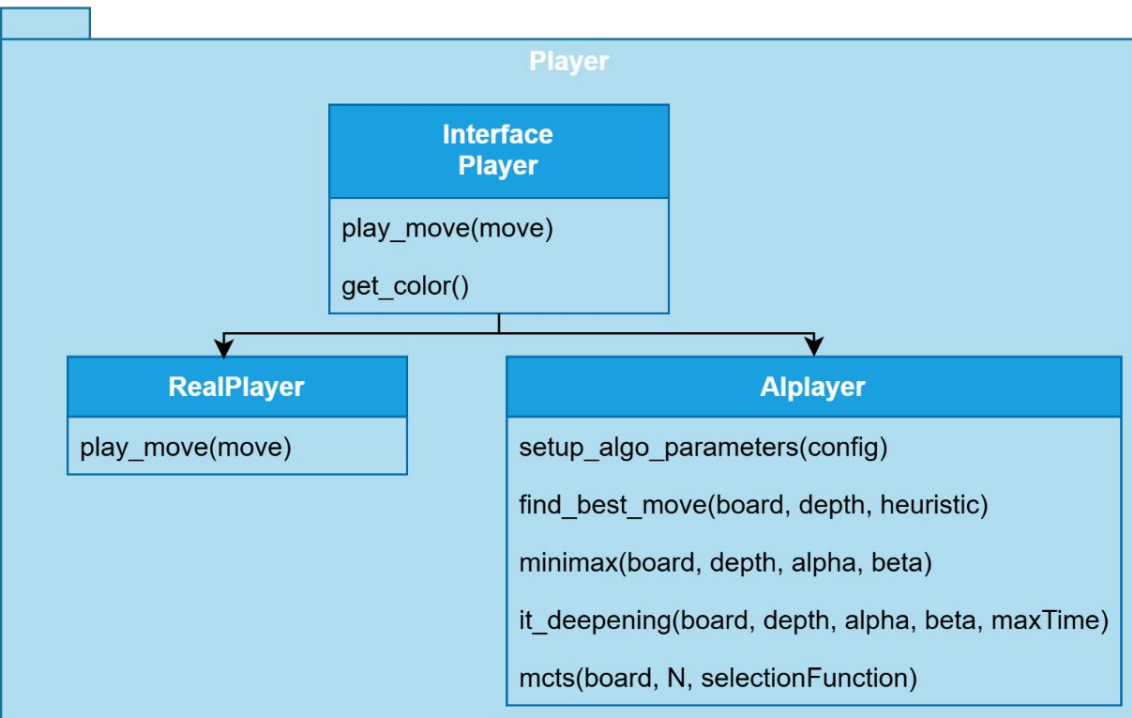
HistoryManager

push(move,stack)

pop(stack)

show_history()

clear_stack(stack)



User Interface

Interface View

```
start()
render(board)
get_input()
```

CLIShell

```
clear_terminal()
autocompletion_loop()
```

GUIApp

```
animate_move(move)
click_to_board_location()
```

ShortcutManager

```
register(key, callback)
```

Network

GameRouter

```
process_move(move)
new_invitation(playerid)
manage_status(playerid,up)#highlev
handle_accept/decline/cancel()
start/stop_game(playerid1,playerid2)
handle_timeout_invitation()
handle_game_over()
timeoutafkig()
send_error(playerid,error)
```

GameClient

```
join(ip, port)
server list
ping
send_new(playerid)
send_accept/decline/cancel()
send_away/back
request_scoreboard()
request_server_status()
quit()
```

GameServer

```
start_server(port,is_daemon))
stop_server()
get_server_status()
timeouted()
```

DiscoveryService

```
broadcast_presence()
broadcast_shutdown()
(server_name, port)
handle_ping() #pong
listen_for_servers()
clear_inactive_servers()
update_local_list()
```

Player Manager

```
get_players()
get_scoreboard()
get_player_details(id)
update_state(id, status)#lowlev
set_scoreboard()
```

ID	Phase	Semaine	Tâche (Task)	Responsables
1	CONCEPTION	S0-S1	Analyse Sujet + Architecture (Draw.io) + Git Setup	TOUS
2	CONCEPTION	S2	Algorithmes et Structures de Données + Brouillon Rapport Préliminaire + Pseudo-code IA	TOUS
3	CONCEPTION	S3	Maquettes GUI (Papier/Figma) & Pseudo-code IA & RAPPORT + ORAL PRELIMINAIRE	TOUS
4	DEV CORE	S4	DÉBUT DU CODE: Initialisation Programme & Execution [F1, F2, F3]	Greg & Rémi
5	DEV CORE	S4	DÉBUT DU CODE: Structures & BitboardOps (Macros) [F26, F27]	Gab & Matthieu
6	DEV CORE	S4	DÉBUT DU CODE: GameState & Init Board [F20, F24]	Arthur
T1	TEST UNITAIRES	S4	Mise en place framework de tests (ex: GTest) & Tests des Bitboard Ops Documentation	TOUS
7	DEV CORE	S5	Rules: get_legal_moves & compute_flips (CRITIQUE) [F10, F11, F27]	Gab & Rémi
8	DEV CORE	S5	CLI Basic : Boucle de jeu texte pour tester le moteur & Affichage [F4, F9, F17, F19]	TOUS
T2	TEST UNITAIRES	S5	Tests unitaires du Moteur de Règles (get_legal_moves, compute_flips) Documentation	TOUS
9	MILESTONE	S5	JALON 1: MOTEUR FONCTIONNEL (CLI Basique)	TOUS
10	DEV CORE	S6	CLI Basic : Sauvegarde, Undo/Redo & Shell [F12, F15, F16, F18, F21, F22, F23, F25]	Greg
11	DEV SQUADS	S6	IA: Stratégie Random & Greedy [F8, F13]	Squad IA
T3	TEST UNITAIRES	S6	Tests unitaires des IA (Random & Greedy) Documentation	Squad IA
12	DEV SQUADS	S6	GUI: Fenêtre vide & Affichage Plateau statique [F7, F28]	Squad GUI
13	DEV SQUADS	S6	NET: Sockets Serveur & Client (Base) [F38]	Squad Net
T4	TEST UNITAIRES	S6	Tests unitaires des Sockets & Connexion de base Documentation	Squad Net
14	DEV SQUADS	S7	IA: Minimax (Algo de base) [F31, F32]	Squad IA
T5	TEST UNITAIRES	S7	Tests unitaires de l'IA Minimax Documentation	Squad IA
15	DEV SQUADS	S7	GUI: Gestion Clics Souris & Input [F29, F30]	Squad GUI
16	DEV SQUADS	S7	NET: Échange de coups (Protocole complet) [F39]	Squad Net
17	DEV SQUADS	S8	IA: Alpha-Beta & Heuristiques [F32, F33, F34, F35]	Squad IA
T6	TEST UNITAIRES	S8	Tests unitaires de l'IA Alpha-Beta et des fonctions Heuristiques Documentation	Squad IA
18	DEV SQUADS	S8	GUI: Intégration Réseau dans le Graphique [F28, F40]	Squad GUI + Net
T7	TEST UNITAIRES	S8	Tests unitaires du Protocole Réseau d'échange de coups Documentation	Squad Net
19	DEV SQUADS	S9	IA : MCTS & ML [F36, F37]	Squad IA
20	DEV SQUADS	S9	Mode Tournoi & Tests Charge [F5, F6, F14]	Squad Net
T8	TEST UNITAIRES	S9	Tests unitaires de l'IA MCTS & ML Documentation	Squad IA
21	MILESTONE	S10	JALON 2: FEATURE COMPLETE (BETA)	TOUS
22	POLISH	S10	Animations GUI & Polish Visuel	Squad GUI
23	POLISH	S11	Optimisation Code & Fuites Mémoire	TOUS
T9	POLISH	S11	Revue et correction des tests après optimisation du code	TOUS
24	FINAL	S12	Revue Documentation (Doxygen) & Rapport Final	TOUS
25	FINAL	S14	Soutenance Finale	TOUS

Réseaux

